SQL SERVER · T-SQL AVANÇADO

BANCO DE DADOS · AULA 08

Segurança e Boas Práticas em Banco de Dados

Controle de Acesso, Permissões e Proteção contra Vulnerabilidades
Organização, Manutenção e Documentação de Código T-SQL

```
seguranca.sql — VendasDB

-- Princípio do menor privilégio
CREATE USER app_userWITHOUT LOGIN
GRANT EXECUTE ON SCHEMA app TO app_user

-- Usuário não tem SELECT direto
-- Apenas via stored procedures
ALTER DATABASEVendasDB
SET QUERY_STORE ON
```

DEFENSE IN DEPTH · ZERO TRUST

8.1 · DEFESA ATIVA | 8.2 · QUALIDADE SUSTENTÁVEL

Agenda da Aula

Dois blocos cobrindo defesa ativa e qualidade sustentável em produção

8.1

BLOCO I

Controle de Acesso, Permissões e Proteção contra Vulnerabilidades

Princípios de segurança · Autenticação (Windows vs SQL) · Logins e usuários · Roles de usuários · Roles de servidor e database · Permissões granulares (GRANT/DENY/REVOKE) · Schemas e ownership chaining · SQL Injection — causa e causa e prevenção · TDE (Transparent Data Encryption) · Always Encrypted · Row-Level Row-Level Security · Dynamic Data Masking · Auditoria

~55 min

12 TÓPICOS

8.2

BLOCO II

Organização, Manutenção e Documentação de Código T-SQL T-SQL

Convenções de nomenclatura · Estrutura de projetos · Versionamento (migrations) · Comentários e documentação · sp_addextendedproperty · Modularização · Code review · Testes automatizados · CI/CD para database · Padrões de qualidade · Checklist profissional

~45 min

11 TÓPICOS

TOTAL ESTIMADO: 100 MINUTOS · 20 SLIDES · 15+ EXEMPLOS DE CÓDIGO T-SQL REPRODUZÍVEIS

→ 8.1 COMEÇA AQUI

CAPÍTULO

8.1

DEFENSE IN DEPTH

Controle de Acesso, Permissões e Proteção contra Vulnerabilidades

Defesa em profundidade — camadas que protegem dados sensíveis em ambientes em ambientes regulados

- Princípios
- Autenticação
- Logins e usuários
- Roles
- Permissões
- Schemas
- SQL Injection
- TDE
- Always Encrypted
- Row-Level Security
- Dynamic Data Masking
- Auditoria

Princípios de Segurança em Banco de Dados

Seis pilares que devem guiar toda decisão de segurança

01 · LEAST PRIVILEGE
Menor Privilégio

Cada usuário/login recebe apenas as permissões estritamente necessárias para sua função. Aplicação web Aplicação web não precisa de DDL. Relatório não precisa precisa de UPDATE. Auditor não precisa de INSERT. Conceder pouco, negar muito.

02 · DEFENSE IN DEPTH
Defesa em Profundidade

Múltiplas camadas de proteção. Se uma falha, outras seguram. Exemplos: firewall → autenticação → autorização → criptografia → auditoria. Atacante precisa vencer todas para acessar dados.

03 · NO SECURITY BY OBSCURITY
Segurança por Obscuridade NÃO

Esconder tabela não é protegê-la. Senha hardcoded em SP não é criptografia. Offuscar nome de coluna não confunde atacante. Usar mecanismos reais: GRANT/DENY, TDE, Always Encrypted.

04 · FAIL-SAFE DEFAULTS
Fail-Safe Defaults

Default deve ser NEGAR acesso. Explicitamente conceder o que for necessário. Exemplo: CREATE USER sem permissões — adicionar GRANT conforme necessário, nunca conceder tudo e depois negar seletivamente.

05 · ECONOMY OF MECHANISM
Economia de Mecanismo

Manter simples e uniforme. Um mecanismo de autenticação (SSO/Azure AD), não dez. Uma role por perfil, não cem. Simplicidade reduz erro de configuração — fonte nº1 de vulnerabilidades.

06 · COMPLETE AUDIT
Auditoria Completa

Toda ação sensível deve ser registrada. Quem acessou o quê, quando, de onde. Logs em local protegido, imutáveis. SQL Audit, Extended Events, ou solução SIEM. Sem auditoria, não há como investigar incidentes.

Regra de ouro: confiança zero (Zero Trust). Verificar sempre, nunca assumir que usuário é confiável.

Autenticação no SQL Server

Três modos — cada um com caso de uso específico. Windows Authentication é o padrão recomendado.

3 MODOS | PADRÃO · WINDOWS

Característica	✦Windows Authentication	⚿SQL Server Authentication	☁Azure AD Authentication
Tipo	Integrated (Kerberos/NTLM)	SQL Login (user/pass)	Azure AD (OAuth/OIDC)
Credenciais	Windows Active Directory	Próprias do SQL Server	Azure AD (cloud)
Senha no SQL	✕ Não (usa Windows)	● Sim (hash no SQL)	✕ Não (usa Azure AD)
Default?	✓ Sim (recomendado)	⚠ Só se necessário	✓ Azure SQL Database
Auditoria	✓ Windows Event Log	⚠ SQL Server log	✓ Azure AD log
MFA	⚠ Via Windows	✗ Não suporta	✓ Suporte nativo
Password policy	⚠ Política Windows	✓ CHECK_POLICY = ON	✓ Azure AD policy
Centralização	✓ Active Directory	✗ Por servidor	✓ Azure AD (global)
Aplicação web	⚠ Delegation (Kerberos)	✓ Simples	✓ Via Managed Identity
Limitações	Requer AD; não funciona em Linux client sem configuração	Senha fraca é comum; ataques de força bruta	Requer Azure AD; latência de rede
Recomendado para	Ambientes Windows corporativos	Apps legados, cenários mistos	Azure SQL, cloud-native
Configuração	sp_addlogin dispensado; use CREATE LOGIN	CREATE LOGIN … WITH PASSWORD	CREATE LOGIN … FROM EXTERNAL PROVIDER

◆ **Regra geral:** Windows Authentication quando possível. Azure AD para cloud. SQL Authentication apenas para casos legados.

MODELO DE SEGURANÇA

Logins, Usuários e Roles

O SQL Server tem modelo de dois níveis: Logins (servidor) e Usuários (database). Logins Logins autenticam no servidor; Usuários autorizam no database. Roles agrupam permissões permissões para simplificar gestão.

CONCEITOS

- **Login** — entidade de servidor (autenticação)
- **User** — entidade de database (autorização)
- **Role** — grupo de permissões
- Um login pode estar mapeado para múltiplos usuários (um por database)

TIPOS DE ROLE

- **Server Roles** — fixas (sysadmin, securityadmin, dbcreator, etc.)
- **Database Roles** — fixas (db_owner, db_datareader, db_datawriter, db_ddladmin) e customizadas
- **Application Roles** — para apps (ativadas com senha)

HIERARQUIA

Login → User (em cada database) → Role (membership) → Permissions

PERMISSÕES

- `GRANT` — concede
- `DENY` — nega explicitamente (sobrescreve GRANT)
- `REVOKE` — remove concessão/negação

BOAS PRÁTICAS

- Usar roles em vez de conceder a usuários individuais
- Negar (DENY) é mais forte que não conceder
- Aplicação usa login dedicado (não SA)
- Senha forte + expiração

logins_roles.sql — VendasDB — T-SQL

```sql
-- 1. Criar login (servidor)
CREATE LOGIN app_user
WITH PASSWORD = 'SenhaForte#2026!'
    CHECK_POLICY = ON   -- política de senha do Windows
    CHECK_EXPIRATION = ON   -- expira senha
    DEFAULT_DATABASE = VendasDB
GO

-- 2. Criar usuário no database
USE VendasDB
GO
CREATE USER app_user FOR LOGIN app_user
GO

-- 3. Criar role customizada
CREATE ROLE rl_app_vendas
GO

-- 4. Conceder permissões à role
GRANT SELECT, INSERT, UPDATE, EXECUTE
ON SCHEMA::dbo TO rl_app_vendas
-- Nega DELETE e DDL
DENY DELETE ON SCHEMA::dbo TO rl_app_vendas
DENY ALTER ON SCHEMA::dbo TO rl_app_vendas
GO

-- 5. Adicionar usuário à role
ALTER ROLE rl_app_vendas ADD MEMBER app_user
GO

-- 6. Ver permissões efetivas
SELECT
    permission_name,
    state_desc,
    class_desc,
    OBJECT_NAME(major_id) AS Objeto
FROM sys.database_permissions dp
JOIN sys.database_principals p
    ON p.principal_id = dp.grantee_principal_id
WHERE p.name = 'app_user';
GO

-- 7. Verificar permissão efetiva em objeto
EXECUTE AS USER = 'app_user';
SELECT * FROM fn_my_permissions('dbo.Clientes', 'OBJECT')
WHERE permission_name IN ('SELECT', 'INSERT', 'UPDATE', 'DELETE');
REVERT;
GO

-- 8. Database roles fixas (usar com cautela)
-- db_owner      → controle total (NUNCA para apps)
-- db_datareader → SELECT em todas as tabelas
-- db_datawriter → INSERT/UPDATE/DELETE em todas

-- 9. Server roles fixas
```

◆ **Padrão produção:** aplicação conecta com login dedicado membro de role customizada — nunca `db_owner`, nunca `sysadmin`.

ORGANIZAÇÃO E SEGURANÇA

Schemas e Ownership Chaining

Schemas não são apenas organização lógica — são fundamentais para segurança. Ownership segurança. Ownership Chaining permite que uma SP acesse tabelas sem que o caller tenha caller tenha permissão direta. É a base do padrão de segurança em produção.

SCHEMAS

- Container lógico de objetos (tabelas, views, SPs, functions)
- Default: `dbo` (database owner)
- Customizados: `app`, `audit`, `report`, etc.
- Permissões podem ser concedidas por schema
- Separa lógica de negócio (`dbo.Clientes`, `app.usp_Vendas`, `audit.Log`)

OWNERSHIP CHAINING

- Se SP e tabela têm mesmo owner, SQL Server não checa permissão na tabela
- Apenas checa permissão de `EXECUTE` na SP
- Permite cenário: `app_user` só tem `EXEC`, não `SELECT`
- Resolve problema de dar SELECT direto para app

QUANDO NÃO FUNCIONA

- SP e tabela em databases diferentes (sem `TRUSTWORTHY`)
- SP e tabela com owners diferentes
- Dynamic SQL (`sp_executesql`) — ownership chaining não se aplica

BOAS PRÁTICAS

- Schemas separados por responsabilidade (`dbo`, `app`, `audit`, `report`)
- SPs no schema `app`, tabelas em `dbo`
- Conceder `EXECUTE` em `app`, negar `SELECT` em `dbo`
- Documentar em `sp_addextendedproperty`

schemas_ownership_chaining.sql — T-SQL · 2 COLS

```sql
-- 1. Criar schemas separados
CREATE SCHEMA app    -- stored procedures da aplicação
CREATE SCHEMA audit  -- tabelas de auditoria
CREATE SCHEMA report -- views de relatório
GO

-- 2. Tabelas em dbo (dados)
CREATE TABLE dbo.Clientes
    ClienteId INT PRIMARY KEY
    Nome VARCHAR(100)
    CPF CHAR(11)    -- dado sensível (LGPD)
    Email VARCHAR(150)
    Status VARCHAR(20)

GO

-- 3. SP em schema app
CREATE OR ALTER PROCEDURE app.usp_ClienteObter
   @ClienteId INT
AS
BEGIN
  SET NOCOUNT ON
   -- SP acessa dbo.Clientes — caller NÃO precisa de SELECT
   SELECT ClienteId, Nome, Email, Status
   FROM dbo.Clientes
   WHERE ClienteId = @ClienteId
   -- Notar: CPF não é retornado (mínimo necessário)
END
GO

-- 4. Criar usuário aplicação
CREATE USER app_user WITHOUT LOGIN
-- WITHOUT LOGIN: user para contexto EXECUTE AS, não para conexão
GO

-- 5. Conceder apenas EXECUTE em schema app
GRANT EXECUTE ON SCHEMA::app TO app_user
-- NÃO conceder SELECT em dbo.Clientes
DENY SELECT ON SCHEMA::dbo TO app_user
GO

-- 6. SP sempre executa como app_user
ALTER PROCEDURE app.usp_ClienteObter
   @ClienteId INT
WITH EXECUTE AS 'app_user'
AS
BEGIN
  SET NOCOUNT ON
   -- Ownership chaining: app.usp_* e dbo.Clientes têm mesmo owner
   -- SQL Server NÃO checa SELECT em dbo.Clientes
   SELECT ClienteId, Nome, Email, Status
   FROM dbo.Clientes
   WHERE ClienteId = @ClienteId

   -- Log de auditoria
  INSERT INTO audit.LogAcessos
       (Usuario, Tabela, RegistroId, DataAcesso)
  VALUES
       (ORIGINAL_LOGIN(), 'Clientes', @ClienteId, GETDATE())
END
GO

-- 7. Teste: simular usuário comum
EXECUTE AS USER = 'app_user'
-- ❌ Tentar SELECT direto
SELECT * FROM dbo.Clientes -- ERRO: permissão negada
-- ✅ Executar SP (permitido)
EXEC app.usp_ClienteObter @ClienteId = 1  -- OK
REVERT
GO
```

Padrão nº1 em produção: app conecta com user que só tem `EXECUTE` em SPs — SQL Injection não vira exfiltração de dados.

✦ VULNERABILIDADE Nº1

SQL Injection — Causa e Prevenção

SQL Injection (SQLi) ocorre quando entrada do usuário é concatenada em concatenada em string SQL sem parametrização. Atacante injeta código SQL código SQL malicioso que é executado pelo banco. É a vulnerabilidade nº1 vulnerabilidade nº1 em aplicações web há mais de 20 anos — e ainda causa ainda causa incidentes.

COMO FUNCIONA

- App recebe: ClienteId = 1 OR 1=1
- Concatena: `SELECT * FROM Clientes WHERE ClienteId = 1 OR 1=1`
- Banco executa: retorna TODOS os clientes
- Pior caso: 1; DROP TABLE Clientes; --
- Ou: `1; INSERT INTO Logins VALUES('hacker','admin'); --`

TIPOS DE ATAQUE

- In-band — resultado voltado na mesma query
- Union-based — usa UNION para extrair dados
- Blind — boolean-based (pergunta sim/não)
- Time-based — usa SLEEP() para inferir dados
- Out-of-band — envia dados para servidor externo (DNS, HTTP)

PREVENÇÃO

- sp_executesql com parâmetros (visto Aula 02)
- Stored Procedures com parâmetros (não concatena)
- ORM com parameterized queries (EF, Dapper)
- Validação de entrada (whitelist, não blacklist)
- Princípio do menor privilégio (app sem DDL)
- WAF (Web Application Firewall) — camada extra

> ⚠ MESMO COM SP, CUIDADO
> • SP sem SQL dinâmico é segura
> • SP que concatena string internamente NÃO é segura

✦ ✕ VULNERÁVEL

```
-- ✕✕✕ VULNERÁVEL: concatenação de string
CREATE PROCEDURE usp_LoginVulneravel
  @Usuario VARCHAR(50),
  @Senha VARCHAR(50)
AS
BEGIN
  DECLARE @sql NVARCHAR(4000)
  SET @sql = N'SELECT * FROM dbo.Usuarios '
           + N'WHERE Login = ''' + @Usuario + ''''
           + N' AND Senha = ''' + @Senha + ''''
  EXEC sp_executesql @sql
END
GO

-- Ataque clássico:
EXEC usp_LoginVulneravel
  @Usuario = 'admin'' --',
  @Senha = 'x'' OR ''1''=''1'
-- SQL resultante:
-- SELECT * FROM dbo.Usuarios
-- WHERE Login = 'admin' -- AND Senha = 'x' OR '1'='1'
-- Retorna TODOS os usuários (1=1 é verdadeiro)

-- Ataque devastador:
EXEC usp_LoginVulneravel
  @Usuario = 'admin'' --',
  @Senha = 'x''; DROP TABLE dbo.Usuarios; --'
-- SQL resultante:
-- SELECT * FROM dbo.Usuarios WHERE Login = 'admin' -- AND Senha = 'x'
-- DROP TABLE dbo.Usuarios; --
GO
```

✓ ✅ SEGURO

```
-- ✅✅✅ SEGURO: sp_executesql com parâmetros
CREATE OR ALTER PROCEDURE usp_LoginSeguro
  @Usuario VARCHAR(50),
  @Senha VARCHAR(50)
AS
BEGIN
  SET NOCOUNT ON
  -- Parâmetros são tipados — impossível injetar SQL
  DECLARE @sql NVARCHAR(4000)
  SET @sql = N'SELECT UsuarioId, Login, Nome
  FROM dbo.Usuarios
  WHERE Login = @pUsuario
  AND SenhaHash = HASHBYTES(''SHA2_512'', @pSenha)'

  EXEC sp_executesql @sql,
    N'@pUsuario VARCHAR(50), @pSenha VARCHAR(50)',
    @pUsuario = @Usuario,
    @pSenha = @Senha
END
GO
-- Mesmo se atacante passar @Senha = 'x'' OR ''1''=''1',
-- o SQL Server trata como STRING (não código).
-- Não encontra login → retorna vazio.
GO

-- ✅✅✅ MELHOR AINDA: SP sem SQL dinâmico
CREATE OR ALTER PROCEDURE usp_LoginMelhor
  @Usuario VARCHAR(50),
  @Senha VARCHAR(50)
AS
BEGIN
  SET NOCOUNT ON
  SELECT
    FROM dbo
    WHERE     @Usuario
      AND     HASHBYTES('SHA2_512', @Senha
END
GO
-- Sem SQL dinâmico = sem risco de injection
-- Parâmetros são tratados como VALORES, nunca como CÓDIGO
GO

-- ✅ VALIDAÇÃO DE ENTRADA adicional
CREATE OR ALTER PROCEDURE usp_LoginValidado
  @Usuario VARCHAR(50),
  @Senha VARCHAR(50)
AS
BEGIN
  SET NOCOUNT ON
  -- Whitelist: apenas alfanuméricos em usuário
  IF @Usuario LIKE '%[^a-zA-Z0-9@._-]%'
    THROW 50001, 'Caractere inválido em usuário', 1

  -- Tamanho mínimo de senha
  IF LEN(@Senha) < 8
    THROW 50002, 'Senha deve ter ao menos 8 caracteres', 1

  SELECT
    FROM dbo
    WHERE     @Usuario
      AND     HASHBYTES('SHA2_512', @Senha
END
GO
```

CRIPTOGRAFIA EM REPOUSO

TDE — Transparent Data Encryption

TDE criptografa arquivos de dados (`.mdf`) e log (`.ldf`) em repouso (em disco). Transparente para aplicação — sem mudança de código. Protege contra roubo físico de roubo físico de disco ou backup. **Não protege** dados em memória ou tráfego.

O QUE PROTEGE / NÃO PROTEGE

✓ PROTEGE

- Arquivos .mdf, .ndf, .ldf
- Backups (.bak)
- Snapshot files

✗ NÃO PROTEGE

- Dados em memória (buffer pool) pool)
- Dados em tráfego (rede)
- Acesso via SQL Server (autenticado)

HIERARQUIA DE CHAVES

```
Windows DPAPI
└─ Service Master Key
└─ Master Key (db master)
└─ Certificate (master)
└─ DEK (db alvo)
```

QUANDO USAR

- Dados sensíveis em repouso (LGPD, HIPAA, PCI)
- Discos sem BitLocker/dm-crypt
- Backups armazenados offsite

Limitações: TempDB criptografado automaticamente se qualquer DB tem TDE · TDE · Compressão de backup reduz eficácia · Performance: **2–5% overhead** típico · típico · Backup de DB com TDE não restaura sem certificate.

⚠ **CRÍTICO:** Backup do certificate é OBRIGATÓRIO. **Perder certificate = perder dados.**

✦ IMPLEMENTAÇÃO T-SQL · PASSO A PASSO

```sql
-- 1. Criar Master Key no database master
USE master
GO
CREATE MASTER KEY
ENCRYPTION BY PASSWORD = 'SenhaForte#2026!'
GO

-- 2. Criar Certificate protegido pela Master Key
CREATE CERTIFICATE CertTDE_VendasDB
WITH SUBJECT 'Certificate para TDE do VendasDB'
  EXPIRY_DATE '20271231'
GO

-- 3. Criar Database Encryption Key no database alvo
USE VendasDB
GO
CREATE DATABASE ENCRYPTION KEY
WITH ALGORITHM = AES_256 -- algoritmo mais forte
ENCRYPTION BY SERVER CERTIFICATE CertTDE_VendasDB
GO

-- 4. Habilitar TDE
ALTER DATABASE VendasDB
SET ENCRYPTION ON
GO

-- 5. Verificar status
SELECT
        AS


FROM
LEFT JOIN
    ON
WHERE     'VendasDB'
-- encryption_state: 0=sem key, 1=descriptografado,
--                   2=em progresso, 3=criptografado, 4=alteração
GO

-- 6. ⚠ CRÍTICO: backup do certificate
-- Sem este backup, NÃO é possível restaurar o database
BACKUP CERTIFICATE CertTDE_VendasDB
TO FILE 'C:\Backup\CertTDE_VendasDB.cer'
WITH PRIVATE KEY
  FILE 'C:\Backup\CertTDE_VendasDB.pvk'
  ENCRYPTION BY PASSWORD = 'SenhaForte#2026!'

GO

-- 7. Restaurar em outro servidor (recuperação de desastre)
USE master
GO
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'SenhaForte#2026!'
CREATE CERTIFICATE CertTDE_VendasDB
FROM FILE 'C:\Backup\CertTDE_VendasDB.cer'
WITH PRIVATE KEY
  FILE 'C:\Backup\CertTDE_VendasDB.pvk'
  DECRYPTION BY PASSWORD = 'SenhaForte#2026!'

-- Agora pode restaurar backup de VendasDB
GO

-- 8. Desabilitar TDE (se necessário)
ALTER DATABASE VendasDB SET ENCRYPTION OFF
-- Aguardar descriptografia completar
DROP DATABASE ENCRYPTION KEY
GO
```

CRIPTOGRAFIA PONTA-A-PONTA

Always Encrypted

Always Encrypted criptografa dados no **cliente (driver)** antes de enviar ao SQL Server. Banco nunca Banco nunca vê texto puro. **Mesmo DBA não consegue ler dados sensíveis.** Resolve problema problema clássico: "administrador do banco não deveria ver dados de clientes".

✓ PROTEGE

- Dados em repouso
- Dados em trânsito
- Buffer pool (memória SQL)

✗ NÃO PROTEGE

- Dados no cliente (memória, logs)
- Chaves (gerenciamento crítico)

COMO FUNCIONA — CHAVES

CMK **Column Master Key** — no cliente (Azure Key Vault, Windows Cert Store)

CEK **Column Encryption Key** — no database, criptografada pela CMK

Driver SQL Server faz criptografia/descriptografia transparente. SQL Server recebe ciphertext, nunca plaintext.

TIPOS DE CRIPTOGRAFIA

DETERMINISTIC
Mesmo valor = mesmo ciphertext. Permite
Permite equal join e equality search.

RANDOMIZED
Mais seguro. Não permite busca direta.

QUANDO USAR

- Dados altamente sensíveis (CPF, cartão crédito, dados saúde)
- Cenários com DBA externo (cloud managed)
- Compliance (HIPAA, PCI-DSS, LGPD)

Limitações: Range queries só com DETERMINISTIC · Funções (UPPER, etc.) não funcionam em colunas criptografadas · Tipos suportados limitados (sem TEXT, IMAGE, TIMESTAMP) · Overhead de 10–20% em queries filtradas.

Enclave (SQL 2019+): Permite operações em dados criptografados (LIKE, range). Requer Requer hardware SGX (confidential computing).

✦ SCHEMA T-SQL · INSERT · BUSCA · DBA

```
-- Configuração via SSMS (interface gráfica) ou PowerShell
-- Aqui mostramos o resultado no schema

-- 1. Tabela com colunas criptografadas
CREATE TABLE dbo.
    INT IDENTITY PRIMARY KEY
    VARCHAR(100)  -- não sensível
    -- DETERMINISTIC: permite busca por igualdade e joins
    CHAR(11)
      ENCRYPTED WITH
        ENCRYPTION_TYPE = DETERMINISTIC
      ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
        COLUMN_ENCRYPTION_KEY
    -- RANDOMIZED: mais seguro, sem busca direta
    VARCHAR(150)
      ENCRYPTED WITH
        ENCRYPTION_TYPE = RANDOMIZED
      ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
        COLUMN_ENCRYPTION_KEY
    DECIMAL(18,2)
      ENCRYPTED WITH
        ENCRYPTION_TYPE = RANDOMIZED
      ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
        COLUMN_ENCRYPTION_KEY
    DATETIME2 DEFAULT
GO

-- 2. Insert via aplicação cliente com driver habilitado
-- App usa connection string com:
--   Column Encryption Setting=Enabled;
-- Driver criptografa CPF, Email, Saldo antes de enviar
INSERT INTO dbo.
VALUES ('João Silva', '12345678901', 'joao@email.com', 5000.00)
GO

-- 3. Query — driver descriptografa transparente
-- (com driver habilitado):
SELECT
FROM dbo.
WHERE
-- App vê texto puro; SQL Server vê apenas ciphertext
GO

-- 4. Busca por CPF (DETERMINISTIC permite)
SELECT FROM dbo.  WHERE  '12345678901'
-- Funciona: CPF é deterministic, mesmo valor = mesmo ciphertext
GO

-- 5. Busca por Email (RANDOMIZED não permite)
-- Não funciona:
SELECT FROM dbo.  WHERE  'joao@email.com'
--Erro: encryption_type = RANDOMIZED não suporta equality search
GO

-- 6. DBA tentando ver dados diretamente (sem driver)
SELECT  FROM dbo.
-- Retorna bytes incompreensíveis (ciphertext)
-- DBA não consegue ler dados sensíveis
GO

-- 7. Ver colunas criptografadas
SELECT
      AS
      AS
      AS

      AS
FROM
JOIN      ON
JOIN       ON
LEFT JOIN
   ON
WHERE      IS NOT NULL
GO
```

PROTEÇÃO EM NÍVEL DE LINHA E COLUNA

Row-Level Security e Dynamic Data Masking

RLS (Row-Level Security) permite que cada usuário veja apenas suas linhas. **DDM** **DDM** (Dynamic Data Masking) mascara colunas sensíveis para usuários sem permissão. permissão. Ambos transparentes para aplicação.

RLS

- Filtra linhas por quem executa
- Predicate function + security policy
- Tipos: FILTER e BLOCK
- Disponível desde SQL 2016

DDM

- Mascara colunas sem UNMASK UNMASK
- Tipos: Default, Email, Random, Custom String
- Dado original no banco
- Não substitui criptografia

CASOS DE USO RLS

- Vendedor vê apenas seus clientes
- Gerente vê clientes de sua região
- Multi-tenant: cada tenant vê só seus dados

CASOS DE USO DDM

- CPF mostrado como `***.***.***-**` para call center
- Email mascarado para relatório
- Cartão de crédito mascarado em log

⚠ **Atenção DDM:** Não protege contra queries que agregam dados (COUNT, GROUP (COUNT, GROUP BY). Atacante pode inferir dados via statistics. Para proteção real, proteção real, usar Always Encrypted.

Defense in depth: RLS + DDM + Always Encrypted + TDE = camadas múltiplas.

ROW-LEVEL SECURITY

```
-- ============ ROW-LEVEL SECURITY ============

-- 1. Tabela com coluna de filtro
CREATE TABLE dbo
          INT PRIMARY KEY
           INT  -- controla acesso
          INT
         DECIMAL 18 2

GO

-- 2. Função predicate (determina quem vê o quê)
CREATE SECURITY dbo fn_VendedorFilter
@VendedorId INT
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN
    -- Vendedor vê apenas suas vendas
    SELECT AS
    WHERE @VendedorId CAST SESSION_CONTEXT VendedorId INT
     -- Gerente (USER_NAME sem nome) vê tudo
     OR USER_NAME  'gerente_vendas'

GO

-- 3. Política de segurança aplica o filtro
CREATE SECURITY dbo POLICY
ADD FILTER PREDICATE dbo fn_VendedorFilter
  ON dbo
ADD BLOCK PREDICATE dbo fn_VendedorFilter
  ON dbo  AFTER INSERT
WITH      ON          OFF
GO

-- 4. Aplicação seta contexto antes de query
EXEC sp_set_session_context
    @key N'VendedorId'  @value 42
SELECT FROM dbo       -- vê apenas suas vendas
GO

-- 5. Gerente vê tudo
EXECUTE AS USER 'gerente_vendas'
SELECT FROM dbo       -- vê todas
REVERT
GO
```

DYNAMIC DATA MASKING

```
-- ============ DYNAMIC DATA MASKING ============

-- 1. Tabela com máscaras
CREATE TABLE dbo
          INT PRIMARY KEY
       VARCHAR 100
       CHAR 11  MASKED WITH FUNCTION 'default()'
        VARCHAR 150  MASKED WITH FUNCTION 'email()'
        DECIMAL 18 2  MASKED WITH FUNCTION 'random(1, 100)'
         VARCHAR 16  MASKED WITH
       FUNCTION 'partial(2, "****", 2)'  -- ex: 45****12

GO

-- 2. Insert dados
INSERT INTO dbo
 VALUES 1  'João Silva'  '12345678901'
        'joao@email.com'  5000.00  '4539123456789012'
GO

-- 3. Usuário SEM UNMASK vê mascarado
CREATE USER  WITHOUT LOGIN
  GRANT SELECT ON dbo  TO
  -- NÃO concede UNMASK
GO

EXECUTE AS USER 'call_center'
SELECT FROM dbo
-- Output:
-- ClienteId: 1
-- Nome:       João Silva
-- CPF:        XXX-XX-XXXX  (default mask)
-- Email:      j***@email.com  (email mask)
-- Saldo:      73.42  (random entre 1 e 100)
-- Cartao:   45****12  (partial)
REVERT
GO

-- 4. Usuário COM UNMASK vê original
GRANT UNMASK TO
EXECUTE AS USER 'gerente_atendimento'
SELECT FROM dbo
-- Output: CPF, Email, Saldo, Cartao — valores reais
REVERT
GO

-- 5. Alterar máscara de coluna existente
ALTER TABLE dbo
ALTER COLUMN  ADD MASKED FUNCTION 'partial(3, "*****", 0)'
-- Mostra: 123*****
GO

-- 6. Remover máscara
ALTER TABLE dbo
ALTER COLUMN DROP MASKED
GO
```

✓ OBSERVABILIDADE E COMPLIANCE

Auditoria e SQL Audit

Auditoria registra **quem** acessou **o quê**, **quando**, **de onde**. Em ambientes regulados (LGPD, regulados (LGPD, SOX, PCI-DSS, HIPAA), é requisito legal. SQL Audit é a ferramenta nativa ferramenta nativa para captura sistemática.

NÍVEIS DE AUDITORIA

- Server-level — captura eventos do servidor (logins, falhas, role changes) changes)
- Database-level — captura eventos de database (SELECT, INSERT, DDL)
- Audit Specification — define quais eventos capturar

FERRAMENTAS

SQL Server Audit nativo EE | Extended Events granular

CDC mudanças em dados | Change Tracking leve, IDs

Temporal Tables histórico | Trigger manual

DESTINOS DO SQL AUDIT

- File (.sqlaudit) — recomendado para produção
- Windows Application Log
- Windows Security Log (requer configuração)

EVENTOS COMUNS A AUDITAR

- LOGIN_SUCCEEDED, LOGIN_FAILED
- SUCCESSFUL_LOGIN_CLASS
- DATABASE_ROLE_MEMBER_CHANGE
- SCHEMA_OBJECT_CHANGE (DDL)
- SELECT, INSERT, UPDATE, DELETE em tabelas sensíveis
- BACKUP, RESTORE · PERMISSION_CHANGE

Boas práticas: Logs em local protegido (imutável) · Retenção conforme compliance compliance (LGPD: 6 meses a 1 ano) · Monitorar tamanho (auditoria gera muito dado) · muito dado) · Automatizar análise (enviar para SIEM).

⚙ 1) CRIAR AUDIT + SPECS

```
-- 1. Criar SQL Server Audit (server-level)
USE master
GO
CREATE SERVER AUDIT Auditoria_VendasDB
TO FILE
  FILEPATH = 'C:\Audit\'
  MAXSIZE = 100 MB   -- rotação
  MAX_ROLLOVER_FILES = 100
  RESERVE_DISK_SPACE = OFF

WITH
  ON_FAILURE = CONTINUE -- não derruba
  QUEUE_DELAY = 1000 -- 1s flush

GO

-- Habilitar
ALTER SERVER AUDIT Auditoria_VendasDB
WITH (STATE = ON)
GO

-- 2. Database Audit Specification
USE VendasDB
GO
CREATE DATABASE AUDIT SPECIFICATION AuditSpec_VendasDB_Sensivel
FOR SERVER AUDIT Auditoria_VendasDB
ADD
   -- SELECT/INSERT/UPDATE/DELETE em tabela sensível
   SELECT, INSERT, UPDATE, DELETE
   ON dbo.Clientes BY public

ADD
   -- DDL em schema dbo
   SCHEMA_OBJECT_CHANGE_GROUP ON SCHEMA::dbo BY public

ADD
   -- mudanças de permissão
   DATABASE_PERMISSION_CHANGE_GROUP BY public

ADD
   -- mudanças de role
   DATABASE_ROLE_MEMBER_CHANGE_GROUP BY public

WITH (STATE = ON)
GO

-- 3. Server Audit Specification
USE master
GO
CREATE SERVER AUDIT SPECIFICATION ServerSpec_Login
FOR SERVER AUDIT Auditoria_VendasDB
ADD
   SUCCESSFUL_LOGIN_GROUP
   FAILED_LOGIN_GROUP -- ataque!
 LOGOUT_GROUP
   SERVER_ROLE_MEMBER_CHANGE_GROUP
   SERVER_PERMISSION_CHANGE_GROUP

WITH (STATE = ON)
GO
```

🔍 2) LER + DETECTAR + TRIGGER

```
-- 4. Ler eventos de auditoria
SELECT
                                    AS
                                  AS User

          AS

FROM   fn_get_audit_file
   'C:\Audit\Auditoria_VendasDB*.sqlaudit'
  DEFAULT, DEFAULT
WHERE      'VendasDB'
ORDER BY      DESC
GO

-- 5. Detectar força bruta (logins falhados)
SELECT
                AS

   COUNT AS
MIN        AS
   MAX        AS
FROM   fn_get_audit_file
   'C:\Audit\Auditoria_VendasDB*.sqlaudit'
  DEFAULT, DEFAULT
WHERE        'LGIF' -- Login Failed
  AND        DATEADD(HOUR, -1, GETDATE
GROUP BY
HAVING COUNT    5 -- >5 tentativas
ORDER BY      DESC
GO

-- 6. Trigger de auditoria customizada (controle fino)
CREATE TABLE dbo
        INT IDENTITY PRIMARY KEY
      SYSNAME
        CHAR(1)   -- I/U/D
        NVARCHAR(50)
          SYSNAME
        NVARCHAR(MAX)
       NVARCHAR(MAX)
      SYSNAME
       NVARCHAR(128)
       NVARCHAR(128)
          DATETIME2 DEFAULT

GO
-- Trigger genérica que loga mudanças em dbo.Clientes
-- (ver Aula 04 para implementação completa)
GO
```

CAPÍTULO

8.2

SEGUNDA PARTE · ORGANIZAÇÃO E DOCUMENTAÇÃO

SECTION HEADER

Organização, Manutenção e Documentação de Código T-SQL

Código que se mantém legível e evolutivo ao longo de anos — não apenas funcional funcional hoje.

TÓPICOS DO BLOCO — 11 ITENS

01 Convenções de nomenclatura	02 Estrutura de projetos	03 Versionamento
04 Comentários	05 `sp_addextendedproperty`	06 Modularização
07 Code review	08 Testes	09 CI/CD
10 Padrões de qualidade	11 Checklist	

BLOCO 8.1 → CONCLUÍDO
BLOCO 8.2 → INICIANDO

BLOCO 8.2 · QUALIDADE SUSTENTÁVEL

PADRÕES DE NAMING

Convenções de Nomenclatura

Convenções consistentes reduzem erro e facilitam busca. Em times grandes, grandes, permitem que qualquer desenvolvedor entenda código de qualquer qualquer colega sem explicação.

PRINCÍPIOS

- Clareza sobre brevidade — `ClienteId` > `cli_id`
- Inglês ou português? — definir no time e ser consistente
- Sem abreviações obscuras — `Quantidade` > `qtde` (às vezes), nunca `q`
- Sem prefixo de tipo — `Nome` > `vNome` (notação húngara é anti-pattern)
- Plural para tabelas, singular para colunas — `dbo.Clientes`, `Clientes.Nome`

PADRÕES POR OBJETO

- Tabelas: `dbo.NomePlural` (Clientes, Pedidos, ItensPedido)
- Views: `vw_NomeDescritivo`
- Stored Procedures: `usp_VerbosObjeto` (usp_ObterCliente, usp_InserirPedido)
- Functions: `fn_NomeDescritivo` (fn_CalcularDesconto)
- Triggers: `trg_AcaoObjeto` (trg_AuditarCliente)
- Índices: `IX_Tabela_Colunas` (IX_Clientes_Cidade)
- Constraints: `PK_Tabela`, `FK_Tabela_TabelaPai`, `CK_Tabela_Regra`, `UQ_Tabela_Coluna`
- Schemas: `dbo` (dados), `app` (SPs), `audit` (logs), `report` (views)

ANTI-PATTERNS

- `sp_Cliente` (sp_ é reservado Microsoft)
- `tbl_Clientes` (prefixo de tipo é redundante)
- `Clientes_2024` (ano no nome — particionar em vez disso)
- `Clientes_Temp` (tabela temporária usa # ou ##)
- `cli_nm` (abreviação obscura)
- `CLIENTES` (uppercase — confunde com keyword)

```sql
-- ✅ PADRÃO RECOMENDADO
-- Tabelas em dbo, no plural
CREATE TABLE dbo.Clientes (
-- Colunas em singular, sem prefixo
ClienteId INT IDENTITY(1,1) NOT NULL,
Nome VARCHAR(100) NOT NULL,
Email VARCHAR(150) NULL,
DataCadastro DATETIME2 NOT NULL DEFAULT(GETDATE()),
-- Constraints nomeadas explicitamente
CONSTRAINT PK_Clientes PRIMARY KEY (ClienteId),
CONSTRAINT UQ_Clientes_Email UNIQUE (Email),
CONSTRAINT CK_Clientes_Email CHECK (
Email LIKE '%_@_%._%'
)
);
GO
-- Índice com nome descritivo
CREATE NONCLUSTERED INDEX IX_Clientes_Email
ON dbo.Clientes (Email);
GO
-- FK nomeada
ALTER TABLE dbo.Pedidos
ADD CONSTRAINT FK_Pedidos_Clientes
FOREIGN KEY (ClienteId) REFERENCES dbo.Clientes(Clie
GO
```

```sql
-- SP com verbo + objeto
CREATE OR ALTER PROCEDURE
dbo.usp_ObterCliente
@ClienteId INT
AS
BEGIN
SET NOCOUNT ON;
SELECT * FROM dbo.Clientes
WHERE ClienteId = @ClienteId;
END
GO
-- Function com fn_
CREATE FUNCTION dbo.fn_CalcularIdade
(@DataNascimento DATE)
RETURNS INT
AS
BEGIN
RETURN DATEDIFF(YEAR, @DataNascimento,
GETDATE());
END
GO
-- Trigger com trg_
CREATE TRIGGER dbo.trg_AuditarCliente
ON dbo.Clientes
AFTER UPDATE
AS
BEGIN
SET NOCOUNT ON;
INSERT INTO dbo.Auditoria (Tabela, Operacao,
 Data)
VALUES ('Clientes', 'U', GETDATE());
END
GO
```

COMPARAÇÃO RÁPIDA

✗ Ruim	✓ Bom
sp_Cliente	usp_ObterCliente
tbl_Clientes	dbo.Clientes
cli_nm	Nome
Clientes_2024	dbo.Clientes (partition)
idx1	IX_Clientes_Email
FK_1	FK_Pedidos_Clientes
chk1	CK_Clientes_Email
t_TempClientes	#TempClientes (temp real)

CONTROLE DE VERSÃO

Versionamento e Migrations

Database changes devem ser versionados como código de aplicação. Migrations (scripts versionados (scripts versionados aplicados em sequência) é o padrão moderno. Permite rollback, auditoria, e auditoria, e deploy automatizado.

TIPOS DE MUDANÇA

- DDL — CREATE/ALTER/DROP (schema)
- DML — INSERT/UPDATE/DELETE (data reference)
- Procedural — SPs, functions, triggers
- Security — users, roles, permissions

PADRÃO DE NOMENCLATURA

- V001__criar_tabela_clientes.sql
- V002__adicionar_coluna_cpf.sql
- V003__criar_indice_email.sql
- V004__criar_sp_obter_cliente.sql
- Versão: número sequencial (V001, V002...) · Descrição: separada por __

FERRAMENTAS

- Flyway (Boomerang) — Java, amplamente usado
- Liquibase — XML/YAML/SQL, multi-SGBDs
- DbUp — .NET · RoundhousE — .NET
- Azure DevOps / GitHub Actions — CI/CD
- SQL Server Data Tools (SSDT) — Visual Studio

ESTRUTURA DE PASTA TÍPICA

- db/migrations/ — scripts versionados
- db/static/ — dados de referência
- db/stored-procedures/ · db/views/ · db/triggers/

PRINCÍPIOS

- Migration é imutável — uma vez aplicada, não alterar
- Sempre idempotente — rodar duas vezes = mesmo resultado
- Sempre com rollback — UP e DOWN
- Testado em dev antes de produção
- Versionado no Git — branch, PR, code review

db/migrations/ V001 → V004 · IDEMPOTENTE

```sql
-- File: db/migrations/V001__criar_tabela_clientes.sql
-- Author: joao.silva · Date: 2026-01-15
-- Description: Schema inicial da tabela Clientes
-- Verifica se já existe (idempotente)
IF NOT EXISTS (
SELECT 1 FROM sys.tables
WHERE name = 'Clientes'
)
BEGIN
CREATE TABLE dbo.Clientes (
ClienteId INT IDENTITY(1,1) NOT NULL,
Nome VARCHAR(100) NOT NULL,
Email VARCHAR(150) NULL,
DataCadastro DATETIME2 NOT NULL
DEFAULT(GETDATE()),
CONSTRAINT PK_Clientes
PRIMARY KEY (ClienteId)
);
CREATE INDEX IX_Clientes_Email
ON dbo.Clientes(Email);
PRINT 'Migration V001 aplicada:
tabela Clientes criada';
END
ELSE
PRINT 'Migration V001 skipped:
tabela já existe';
GO
-- File: V002__adicionar_coluna_cpf.sql
-- Author: maria.santos · Date: 2026-02-20
IF NOT EXISTS (
SELECT 1 FROM sys.columns
WHERE object_id = OBJECT_ID('dbo.Clientes')
AND name = 'CPF'
)
BEGIN
ALTER TABLE dbo.Clientes
ADD CPF CHAR(11) NULL;  -- NULL primeiro
PRINT 'V002 aplicada: coluna CPF adicionada'
END
ELSE
PRINT 'V002 skipped: coluna já existe';
GO
```

```sql
-- File: V003__popular_cpf_e_add_constraint.sql
-- Date: 2026-02-21
-- 1. Popular CPF para clientes existentes
-- (regra de negócio)
UPDATE dbo.Clientes
SET CPF = '00000000000'
-- placeholder, ajustar
WHERE CPF IS NULL;
-- 2. Agora sim, tornar NOT NULL
ALTER TABLE dbo.Clientes
ALTER COLUMN CPF CHAR(11) NOT NULL;
-- 3. Adicionar constraint de uniqueness
IF NOT EXISTS (
SELECT 1 FROM sys.indexes
WHERE name = 'UQ_Clientes_CPF'
)
ALTER TABLE dbo.Clientes
ADD CONSTRAINT UQ_Clientes_CPF UNIQUE (CPF);
GO
-- File: V004__criar_sp_obter_cliente.sql
IF NOT EXISTS (
SELECT 1 FROM sys.objects
WHERE type = 'P' AND name = 'usp_ObterCliente'
)
BEGIN
EXEC('CREATE PROCEDURE dbo.usp_ObterCliente
AS BEGIN SELECT 1 END');
END
GO
ALTER PROCEDURE dbo.usp_ObterCliente
@ClienteId INT
AS
BEGIN
SET NOCOUNT ON;
SELECT ClienteId, Nome, Email, CPF, DataCadastro
FROM dbo.Clientes
WHERE ClienteId = @ClienteId;
END
GO
```

METADADOS

Documentação e sp_addextendedproperty

Documentação viva: metadados que acompanham o código e não ficam obsoletos.

`sp_addextendedproperty` anexa metadados a tabelas, colunas, SPs e outros objetos. SSMS Object Explorer Details.

TIPOS DE DOCUMENTAÇÃO

- Comentários SQL — `--` inline ou `/* */` block
- Extended properties — metadados estruturados
- README em repositório — visão geral
- Diagramas ERD — visualização de schema
- Data dictionary — gerado automaticamente

EXTENDED PROPERTIES

- Caption — nome amigável (ex: "Cliente ID")
- Description — explicação detalhada
- Format — máscara (ex: "###.###.###-##")
- Sensitivity — "Public", "Confidential", "Restricted"
- Source — origem do dado (ex: "Sistema CRM")
- Validation — regra de validação · Examples — exemplos de uso

VANTAGENS

- Documentação vive com o objeto (não em arquivo separado)
- Acessível via T-SQL (`sys.extended_properties`)
- Ferramentas externas podem ler (data dictionary, validar LGPD)
- Visível no SSMS (Object Explorer Details)

ONDE USAR

- Tabelas sensíveis (LGPD: CPF, email, telefone)
- SPs complexas (lógica de negócio)
- Triggers (invisíveis no código app)
- Views calculadas · Constraints não-óbvias

CONSULTAR METADADOS

- `sys.extended_properties`
- `fn_listextendedproperty()`
- SSMS: Object Explorer > Extended Properties

extended_properties.sql

```sql
-- 1. Documentar tabela
EXEC sp_addextendedproperty
    @name = N'Descriptio',
    @value = N'Tabela de clientes ativos. Dados pessoais.',
    @level0type = N'SCHEMA', @level0name = N'dbo',
    @level1type = N'TABLE',  @level1name = N'Clientes';
GO

-- 2. Documentar coluna (LGPD)
EXEC sp_addextendedproperty
    @name = N'Descriptio',
    @value = N'CPF. Dado sensível LGPD Art. 5º.',
    @level0type = N'SCHEMA', @level0name = N'dbo',
    @level1type = N'TABLE',  @level1name = N'Clientes',
    @level2type = N'COLUMN', @level2name = N'CPF';
GO

EXEC sp_addextendedproperty
    @name = N'Sensitivit',
    @value = N'Confidentia',
    @level0type = N'SCHEMA', @level0name = N'dbo',
    @level1type = N'TABLE',  @level1name = N'Clientes',
    @level2type = N'COLUMN', @level2name = N'CPF';
GO

EXEC sp_addextendedproperty
    @name = N'Format',
    @value = N'###.###.###-##',
    @level0type = N'SCHEMA', @level0name = N'dbo',
    @level1type = N'TABLE',  @level1name = N'Clientes',
    @level2type = N'COLUMN', @level2name = N'CPF';
GO

-- 3. Documentar Stored Procedure
EXEC sp_addextendedproperty
    @name = N'Descriptio',
    @value = N'Retorna dados do cliente por ID. Não expõe CPF.',
    @level0type = N'SCHEMA', @level0name = N'dbo',
    @level1type = N'PROCEDURE', @level1name = N'usp_ObterClien
GOte';

EXEC sp_addextendedproperty
    @name = N'Author',
    @value = N'joao.silva@empresa',
    @level0type = N'SCHEMA', @level0name = N'dbo',
    @level1type = N'PROCEDURE', @level1name = N'usp_ObterClien
GOte';

-- 4. Documentar trigger (crítico!)
EXEC sp_addextendedproperty
    @name = N'Descriptio',
    @value = N'Audita mudanças de Status. AFTER UPDATE.',
    @level0type = N'SCHEMA', @level0name = N'dbo',
    @level1type = N'TABLE',  @level1name = N'Clientes',
    @level2type = N'TRIGGER', @level2name = N'trg_AuditarClien
GOte';

-- 5. Atualizar propriedade existente
EXEC sp_updateextendedpro
    @name = N'Descriptio',
    @value = N'Atualizado: retorna também DataNascimento.',
    @level0type = N'SCHEMA', @level0name = N'dbo',
    @level1type = N'PROCEDURE', @level1name = N'usp_ObterClien
GOte';

-- 6. Listar todas as propriedades
SELECT
    major_id, minor_id, class_desc,
    OBJECT_NAME(major_id) AS Objeto,
    COL_NAME(major_id, minor_id) AS Coluna,
    name AS Propriedade, value AS Valor
FROM sys.extended_prope
WHERE major_id IN (
    OBJECT_ID('dbo.Clientes'),
    OBJECT_ID('dbo.usp_ObterCliente')
)
ORDER BY major_id, minor_id, name;
GO

-- 7. Gerar data dictionary completo
SELECT
    s.name AS SchemaName,
    t.name AS TableName,
    t_desc.value AS TableDescript,
    c.name AS ColumnName,
    c_desc.value AS ColumnDescript,
    sens.value AS Sensitivity,
    fmt.value AS Format,
    TYPE_NAME(c.user_type_id) AS DataType
FROM sys.tables t
JOIN sys.schemas s ON s.schema_id
t.schema_id
JOIN sys.columns c ON c.object_id
t.object_id
LEFT JOIN sys.extended_prope t_desc
    ON t_desc.major_id object_id
    AND t_desc.minor_id 0
    AND t_desc.name 'Description'
LEFT JOIN sys.extended_prope c_desc
    ON c_desc.major_id object_id
    AND c_desc.minor_id column_id
    AND c_desc.name 'Description'
ORDER BY s.name, t.name, c.column_id;
GO
```

QUALIDADE DE CÓDIGO

Modularização e Code Review

Código modular é reutilizável, testável e manutenível. Code review é o processo nº1 de garantia de qualidade — captura bugs, captura bugs, dissemina conhecimento, e mantém padrões.

PRINCÍPIOS DE MODULARIZAÇÃO

- DRY (Don't Repeat Yourself) — não duplicar lógica
- Single Responsibility — uma SP = uma função
- Composição — SPs pequenas que se chamam
- Namespacing — schemas separados (app, audit, report)
- Encapsulamento — esconder complexidade em SPs

PADRÕES DE SP

- Tamanho: máximo ~200 linhas
- Parâmetros: máximo 5-7 (acima, refactorar)
- Lógica de negócio em SP, não em app
- Validação no início (fail fast)
- `SET NOCOUNT ON`, `SET XACT_ABORT ON`
- `TRY/CATCH` com log estruturado

CODE REVIEW

- O que revisar: lógica, performance, segurança, padrões
- Quem revisa: outro desenvolvedor (não o autor)
- Quando: antes de merge (PR)
- Ferramentas: GitHub/GitLab/Bitbucket PRs
- Checklist: nomenclatura, SET options, TRY/CATCH, SARGability, índices, segurança

ANTI-PATTERNS

- SP de 1000 linhas (decompor)
- Lógica duplicada em 5 SPs (extrair para função)
- `SELECT *` em produção (colunas explícitas)
- Sem TRY/CATCH em SP transacional
- Concatenar strings em SQL (SQLi)
- Hardcoded values (config em tabela)

usp_ProcessarPedido.sql — MODULAR · TRY/CATCH · SCHEMAS

```sql
-- ❌ ANTI-PATTERN: SP monolítica de 500 linhas
CREATE PROCEDURE dbo.usp_ProcessarTudoRuim
@PedidoId INT
AS
BEGIN
-- 100 linhas de validação
-- 200 linhas de lógica de negócio
-- 100 linhas de auditoria
-- 100 linhas de notificação
END
GO
-- ✅ PADRÃO MODULAR: SPs pequenas e compostas
CREATE OR ALTER PROCEDURE app.usp_ProcessarPedido
@PedidoId INT
AS
BEGIN
SET NOCOUNT ON;
SET XACT_ABORT ON;
BEGIN TRY
BEGIN TRAN;
-- 1. Validar (SP separada)
EXEC app.usp_ValidarPedido @PedidoId;
-- 2. Atualizar estoque (SP separada)
EXEC app.usp_BaixarEstoque @PedidoId;
-- 3. Calcular totais (function separada)
DECLARE @Total DECIMAL(18,2);
SET @Total = dbo.fn_CalcularTotalPedido(@PedidoId);
-- 4. Atualizar pedido
UPDATE dbo.Pedidos
SET Status = 'PROCESSADO',
Total = @Total,
DataProcessamento = GETDATE()
```

CHECKLIST DE CODE REVIEW T-SQL · NO PR

NOMENCLATURA
- ☐ prefixo usp_, usp_, fn_, trg_
- ☐ schema qualification (dbo.nome)
- ☐ (dbo.nome)
- ☐ nome descritivo (verbo + objeto)

ESTRUTURA
- ☐ SET NOCOUNT ON (1ª linha)
- ☐ SET XACT_ABORT ON (transacional)
- ☐ TRY/CATCH com THROW
- ☐ < 200 linhas · < 7 params

PERFORMANCE
- ☐ set-based (sem cursor/loop)
- ☐ SARGable WHERE
- ☐ índices apropriados
- ☐ sem SELECT *

SEGURANÇA
- ☐ sp_executesql (não concat)
- ☐ parâmetros validados
- ☐ sem hardcoded credentials
- ☐ permissões mínimas (role)

MANUTENIBILIDADE
- ☐ comentários de regra de negócio
- ☐ sp_addextendedproperty
- ☐ log estruturado em erro
- ☐ documentação atualizada

TESTES AUTOMATIZADOS

Testes Unitários com tSQLt

tSQLt é framework open-source de testes unitários para T-SQL (estilo JUnit/NUnit). Permite testar SPs, testar SPs, functions, triggers isoladamente. Padrão de equipes maduras em database.

POR QUE TESTAR T-SQL?

- SPs têm lógica de negócio crítica
- Bugs em produção são caros
- Refatoração sem testes é perigosa
- Documentação executável (testes = exemplos)
- CI/CD exige testes automatizados

CARACTERÍSTICAS DO TSQLT

- Open-source (gratuito)
- Instala via script no database
- Sintaxe similar a JUnit/NUnit
- Assertions: `AssertEquals`, `AssertEqualsString`, `AssertObjectExists`
- Test isolation (transação com rollback)
- Gera XML report integrável com CI/CD

ESTRUTURA TÍPICA

- Test Class — schema com prefixo `Test`
- Test Procedure — SP com prefixo `test`
- Setup/Cleanup — executados antes/depois de cada teste

COBERTURA TÍPICA

- SPs de lógica de negócio
- Functions (especialmente com cálculos)
- Triggers (efeitos colaterais)
- Constraints (validação) · Security policies (RLS)

LIMITAÇÕES

- Não testa performance (apenas correção)
- Não substitui testes de integração
- Requer instalação no database

INTEGRAÇÃO CI/CD

- Azure DevOps: tSQLt task
- GitHub Actions: tSQLt action
- Gera relatório JUnit XML

tSQLt_tests.sql

```
-- 1. Instalar tSQLt (uma vez por database)

-- Download em https://tsqlt.org
EXEC tSQLt.Install;
GO

-- 2. Criar Test Class (schema)
EXEC tSQLt.NewTestClass 'TestCliente';
GO

-- 3. SP deve retornar cliente por ID
CREATE PROCEDURE [TestCliente].[test usp_ObterCliente retorna cliente correto]
AS
BEGIN
    -- Arrange: preparar
    INSERT INTO dbo.Clientes (ClienteId, Nome, Email, Status)
    VALUES (1, 'João Teste', 'joao@test.com', 'ATIVO');

    -- Act: executar SP
    DECLARE @Resultado TABLE (ClienteId INT, Nome VARCHAR(100),
        Email VARCHAR(150), Status VARCHAR(20));
    INSERT INTO @Resultado
    EXEC dbo.usp_ObterCliente @ClienteId = 1;

    -- Assert: verificar
    DECLARE @NomeRetorno VARCHAR(100);
    SELECT @NomeRetorno = Nome FROM @Resultado;

    EXEC tSQLt.AssertEqualsString
        @Expected = 'João Teste',
        @Actual = @NomeRetorno,
        @Message = 'Nome deve ser João Teste';
END;
GO

-- 4. SP deve falhar p/ ID inexistente
CREATE PROCEDURE [TestCliente].[test usp_ObterCliente falha para ID inexistente]
AS
BEGIN
    DECLARE @Resultado TABLE (ClienteId INT, Nome VARCHAR(100),
        Email VARCHAR(150), Status VARCHAR(20));
    INSERT INTO @Resultado
    EXEC dbo.usp_ObterCliente @ClienteId = 99999;

    DECLARE @Count INT = (SELECT COUNT(*) FROM @Resultado);

    EXEC tSQLt.AssertEquals
        @Expected = 0,
        @Actual = @Count,
        @Message = 'ID inexistente deve retornar 0';
END;
GO

-- 5. Trigger deve auditar mudança
CREATE PROCEDURE [TestCliente].[test trg_AuditarCliente registra mudança]
AS
BEGIN
    -- Arrange
    INSERT INTO dbo.Clientes (ClienteId, Nome, Email, Status)
    VALUES (1, 'Cliente Teste', 'test@test.com', 'ATIVO');

    DECLARE @AuditAntes INT = (SELECT COUNT(*) FROM dbo.Auditor
        WHERE Tabela = 'Clientes' AND RegistroId = '1');

    -- Act: atualizar Status (dispara trigger)
    UPDATE dbo.Clientes
    SET Status = 'INATIVO'
    WHERE ClienteId = 1;

    -- Assert: deve ter registrado
    DECLARE @AuditDepois INT = (SELECT COUNT(*) FROM dbo.Auditor
        WHERE Tabela = 'Clientes' AND RegistroId = '1');

    EXEC tSQLt.AssertEquals
        @Expected = @AuditAntes + 1,
        @Actual = @AuditDepois,
        @Message = 'Trigger deve registrar auditoria';
END;
GO

-- 6. Executar todos os testes
EXEC tSQLt.RunAll;
-- Output:
-- +----------------------
-- |Test Case Execution  |
-- +----------------------
-- |[TestClientes].[test usp_ObterCliente retorna cliente correto]| ✓
-- |[TestClientes].[test usp_ObterCliente falha para ID inexistente]| ✓
-- |[TestClientes].[test trg_AuditarCliente registra mudança]| ✓
-- +----------+
-- |Summary: 3 | 3 succeeded | 0 failed | 0 errored|
-- +----------+

-- 7. Executar test class específica
EXEC tSQLt.Run 'TestCliente';
GO

-- 8. Gerar relatório XML (para CI/CD)
EXEC tSQLt.XmlResultBin
```

DEVOPS PARA DATABASE

CI/CD para Database

CI/CD (Continuous Integration / Continuous Deployment) para database automatiza testes e deploys. Reduz deploys. Reduz erro humano, garante repetibilidade, e acelera entrega de mudanças.

PIPELINE TÍPICO

1. Developer — escreve migration + testes tSQLt
2. Commit/Push — para branch no Git
3. Pull Request — code review
4. CI build — roda testes automaticamente
5. Merge — para main/develop
6. CD deploy — para dev → staging → produção
7. Smoke tests — verifica pós-deploy

COMPONENTES

- Source control — Git (GitHub, GitLab, Azure DevOps)
- Migration tool — Flyway, Liquibase, DbUp
- Test framework — tSQLt
- CI runner — GitHub Actions, Azure Pipelines, Jenkins
- Artifact — pacote de scripts versionados
- Deploy target — dev, staging, prod databases

PRÁTICAS

- Branch por feature
- PR com code review obrigatório
- Testes automáticos em cada PR
- Deploy em dev automático após merge
- Deploy em staging com aprovação
- Deploy em produção com aprovação manual
- Rollback plan definido

MÉTRICAS

- Tempo de build (< 5 min ideal)
- Cobertura de testes (>70% bom)
- Frequência de deploy · Lead time
- MTTR (Mean Time To Recovery)

.github/workflows/database-ci.yml

GITHUB ACTIONS · 3 JOBS

```yaml
# .github/workflows/database-ci.yml
name: Database CI/CD

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:

  test:
    name: Run tSQLt Tests
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4

      - name: Setup SQL Server
        uses: potatoqualitee/mssqlsuite@v1.7
        with:
          install: sqlengine
          sa-password: TestPass123!

      - name: Install tSQLt
        run: |
          sqlcmd -S localhost -U sa -P TestPass123! -Q "CREATE DATABASE TestDB"
          Get-ChildItem -Recurse -Filter *.sql | Sort-Object FullName | ForEach-Object { sqlcmd -S localhost -U sa -P TestPass123! -d TestDB -i $_.FullName }

      - name: Apply migrations
        run: |
          sqlcmd -S localhost -U sa -P TestPass123! -d TestDB -Q "EXEC tSQLt.RunAll" -o test-results.xml
          Get-Content test-results.xml

      - name: Run tests
        run: |
          flyway -url="jdbc:sqlserver://localhost:1433;databaseName=TestDB" -user=sa -password=TestPass123! -locations=filesystem:db/migrations migrate

      - name: Publish test results
        uses: EnricoMi/publish-unit-test-result-action@v2
        if: always()
        with:
          files: test-results.xml

  deploy-dev:
    name: Deploy to DEV
    needs: test
    if: github.ref == 'refs/heads/develop'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - name: Install Flyway
        run: |
          wget -qO- https://repo1.maven.org/maven2/org/flywaydb/flyway-commandline/9.22.3/flyway-commandline-9.22.3-linux-x64.tar.gz | tar xz
          sudo ln -s flyway-9.22.3/flyway /usr/local/bin

      - name: Deploy migrations
        run: |
          flyway -url="jdbc:sqlserver://dev-sql.empresa.com:1433;databaseName=VendasDB" -user=${{ secrets.DB_DEV_USER }} -password=${{ secrets.DB_DEV_PASS }} -locations=filesystem:db/migrations migrate

  deploy-prod:
    name: Deploy to PROD (with approval)
    needs: deploy-dev
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    environment: production
    steps:
      - uses: actions/checkout@v4

      - name: Install Flyway
        run: |
          wget -qO- https://repo1.maven.org/maven2/org/flywaydb/flyway-commandline/9.22.3/flyway-commandline-9.22.3-linux-x64.tar.gz | tar xz
          sudo ln -s flyway-9.22.3/flyway /usr/local/bin

      - name: Deploy migrations
        run: |
          flyway -url="jdbc:sqlserver://prod-sql.empresa.com:1433;databaseName=VendasDB" -user=${{ secrets.DB_PROD_USER }} -password=${{ secrets.DB_PROD_PASS }} -locations=filesystem:db/migrations migrate

      - name: Smoke tests
        run: |
          sqlcmd -S prod-sql.empresa.com -U ${{ secrets.DB_PROD_USER }} -P ${{ secrets.DB_PROD_PASS }} -d VendasDB -Q "EXEC usp_SmokeTest" -h-1
```

CHECKLIST PROFISSIONAL

Segurança + Qualidade T-SQL

2 CHECKLISTS · 24 ITENS
BLOCO 8.1 + 8.2

✓Checklist de Segurança

AUDITORIA DE DATABASE EM PRODUÇÃO

1. Autenticação Windows preferida
   Não usar SA
2. Logins por aplicação
   Não compartilhar credenciais
3. Roles de servidor/database
   Permissões em roles, nunca em usuários
4. Menor privilégio
   Sem db_owner no app
5. Permissões granulares
   `GRANT/DENY/REVOKE` explícito
6. Schemas segregados
   dbo, app, audit, report
7. Ownership chaining
   Acesso via SPs, sem SELECT direto
8. SQL Injection — prevenção
   `sp_executesql`, nunca concatenar
9. TDE ativo em produção
   Transparent Data Encryption em repouso
10. Always Encrypted
    CPF, cartão — sensíveis ponta-a-ponta
11. RLS + Dynamic Data Masking
    Multi-tenant e dados mascarados
12. Auditoria habilitada
    DDL, logins, acessos a dados sensíveis

✓Checklist de Qualidade T-SQL

CODE REVIEW ANTES DE MERGE

1. Convenções de naming
   `usp_, fn_, trg_, IX_, PK_/FK_/CK_/UQ_`
2. Schema-qualified
   `dbo.nome` em todas as referências
3. SET NOCOUNT ON
   Primeira linha de toda SP
4. SET XACT_ABORT ON
   Em SPs transacionais
5. TRY/CATCH com THROW
   Tratamento de erro robusto
6. Set-based
   Sem cursor/loop desnecessário
7. SARGable WHERE
   Sem função na coluna
8. Sem SELECT *
   Colunas explícitas
9. sp_executesql
   Para SQL dinâmico — nunca concatenar
10. Extended properties
    Documentar tabela/coluna/SP/trigger
11. Testes tSQLt
    SPs críticas com cobertura
12. Migration versionada
    `V001__descricao.sql`, idempotente

DESAFIO FINAL

Auditar database de produção (real ou simulado) aplicando ambos checklists. Documentar gaps e plano de remediação.